Міністерство освіти і науки України
Львівський національний університет імені Івана Франка Факультет електроніки та комп'ютерних технологій

Звіт

Про виконання лабораторної роботи № 5

«Умовні оператори»

Виконав:
Студент групи ФЕП-11с
Полейко М. В.

Перевірив:
асист. Васюта В. М.

Львів - 2026

Мета: Вивчити поняття і застосування умовних операторів у мові програмування С: оператора **if**, конструкції **if – else – if**, оператора **?** та оператора **switch**.

Обладнання та програмне забезпечення

IBM сумісна персональна обчислювальна машина; CLion IDE від JetBrains.

Хід роботи

1. Порівняння двох чисел за допомогою оператора «?»

Завдання. Написати програму, яка видає результат порівняння двох введених з клавіатури чисел за допомогою оператора «?».
Алгоритм.

1. Вводимо з клавіатури два цілих числа a і b.
2. Якщо a > b, виводимо, що перше число більше за друге.
3. Інакше, якщо a < b, виводимо, що перше число менше за друге.
4. Інакше виводимо, що числа рівні.

Порівняння виконується вкладеним тернарним оператором. Функція printf повертає значення типу int, тому її можна використовувати в гілках оператора ?.

Код програми:

```
#include <stdio.h>
int main() {
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d%d", &a, &b);
    //Використовуємо "?" для порівняння
    a > b ? printf("%d is greater than %d\n", a, b):
    a < b ? printf("%d is less than %d\n", a, b):
printf("Numbers are equal\n");
    return 0;
```

Результат виконання:

```
/home/mykolapoleiko/CLionProjects/lab/lab5.1
Enter two numbers: 7 3
7 is greater than 3

Process finished with exit code 0
```

```
/home/mykolapoleiko/CLionProjects/lab/lab5.1
Enter two numbers: 2 9
2 is less than 9

Process finished with exit code 0
```

```
/home/mykolapoleiko/CLionProjects/lab/lab5.1
Enter two numbers: 5 5
Numbers are equal

Process finished with exit code 0
```

2. Визначення типу трикутника

Завдання. Написати програму, яка дозволяє ввести з консолі довжини трьох сторін трикутника (наприклад, 3 4 5) і визначити його тип: а) рівносторонній, рівнобедрений, різносторонній; б) прямокутний, гострокутний, тупокутний.

Алгоритм.

1. Вводимо довжини сторін `a, b, c`.

2. Перевіряємо існування трикутника: усі сторони додатні та сума будь-яких двох сторін строго більша за третю. Якщо умова не виконується, виводимо повідомлення і завершуємо програму.

3. Тип за сторонами: якщо `a == b == c`, трикутник рівносторонній; якщо рівні будь-які дві сторони, рівнобедрений; інакше різносторонній.

4. Тип за кутами: за допомогою обмінів робимо сторону `c` найбільшою (найбільший кут лежить навпроти найбільшої сторони).

Обчислюємо **diff = c² - (a² + b²)**:

- **diff == 0** — прямокутний (теорема Піфагора);
- **diff < 0** — гострокутний;
- **diff > 0** — тупокутний.

Код програми:

```c
#include <stdio.h>

int main() {
    int a, b, c, temporary;
    printf("Enter three sides of triangle: ");
    if (scanf("%d%d%d", &a, &b, &c) != 3) {
        printf("Not a number\n");
        return 1;
    }

    if (a <= 0 || b <= 0 || c <= 0 ||
        a + b <= c || b + c <= a || a + c <= b) {
        printf("Triangle does not exist\n");
        return 1;
        }

    // Класифікація за сторонами
    if (a == b && b == c) {
        printf("1. equilateral triangle\n");
    } else if (a == b || b == c || c == a) {
        printf("1. isosceles triangle\n");
    } else {
        printf("1. scalene triangle\n");
    }

    // Впорядковуємо сторони, щоб c була найбільшою
    if (a > c) { temporary = a; a = c; c = temporary; }
    if (b > c) { temporary = b; b = c; c = temporary; }

    // Класифікація за кутами
    int diff = c * c - (a * a + b * b);
```

```
    if (diff == 0) {
        printf("2. right triangle\n");
    } else if (diff < 0) {
        printf("2. acute triangle\n");
    } else {
        printf("2. obtuse triangle\n");
    }

    return 0;
}
```

Результат виконання:

```
/home/mykolapoleiko/CLionProjects/lab/lab5.2
Enter three sides of triangle: 3 4 5
1. scalene triangle
2. right triangle

Process finished with exit code 0
```

```
/home/mykolapoleiko/CLionProjects/lab/lab5.2
Enter three sides of triangle: 5 5 5
1. equilateral triangle
2. acute triangle

Process finished with exit code 0
```

```
/home/mykolapoleiko/CLionProjects/lab/lab5.2
Enter three sides of triangle: 2 2 3
1. isosceles triangle
2. obtuse triangle

Process finished with exit code 0
```

```
/home/mykolapoleiko/CLionProjects/lab/lab5.2
Enter three sides of triangle: 1 2 3
Triangle does not exist

Process finished with exit code 1
```

3. Виведення тризначного числа прописом

Завдання. Написати програму, яка дозволяє ввести з консолі тризначне число і вивести його прописом.

Алгоритм.

1. Вводимо число **num** і перевіряємо, що воно ціле та належить діапазону від 100 до 999.

2. Виділяємо цифри: сотні **num / 100**, десятки **(num / 10) % 10**, одиниці **num % 10**.

3. За допомогою оператора **switch** виводимо слово для сотень.

4. Якщо цифра десятків дорівнює 1 (числа 10–19), виводимо відповідне слово окремим **switch** за цифрою одиниць; одиниці окремо не виводимо.

5. Інакше виводимо слово для десятків (20–90), а потім слово для одиниць (1–9).

Код програми:

```c
#include <stdio.h>

int main() {
    int num;
    printf("Enter three-digit number: ");
    if (scanf("%d", &num) != 1) {
        printf("Not a number");
        return 1;
    }

    if (num < 100 || num > 999) {
        printf("Number out of range");
        return 1;
    }

    int hundreds = num / 100;
    int tens = (num / 10) % 10;
    int units = num % 10;
```

```
// Сотні
switch (hundreds) {
    case 1: printf("one hundred"); break;
    case 2: printf("two hundred"); break;
    case 3: printf("three hundred"); break;
    case 4: printf("four hundred"); break;
    case 5: printf("five hundred"); break;
    case 6: printf("six hundred"); break;
    case 7: printf("seven hundred"); break;
    case 8: printf("eight hundred"); break;
    case 9: printf("nine hundred"); break;
}

if (tens == 1) {
    // 10-19: одиниці окремо не виводимо
    switch (units) {
        case 0: printf(" ten"); break;
        case 1: printf(" eleven"); break;
        case 2: printf(" twelve"); break;
        case 3: printf(" thirteen"); break;
        case 4: printf(" fourteen"); break;
        case 5: printf(" fifteen"); break;
        case 6: printf(" sixteen"); break;
        case 7: printf(" seventeen"); break;
        case 8: printf(" eighteen"); break;
        case 9: printf(" nineteen"); break;
    }
} else {
    // Десятки (20-90)
    switch (tens) {
        case 2: printf(" twenty"); break;
        case 3: printf(" thirty"); break;
        case 4: printf(" forty"); break;
        case 5: printf(" fifty"); break;
        case 6: printf(" sixty"); break;
        case 7: printf(" seventy"); break;
        case 8: printf(" eighty"); break;
        case 9: printf(" ninety"); break;
    }
```

```
        // Одиниці
        switch (units) {
            case 1: printf(" one"); break;
            case 2: printf(" two"); break;
            case 3: printf(" three"); break;
            case 4: printf(" four"); break;
            case 5: printf(" five"); break;
            case 6: printf(" six"); break;
            case 7: printf(" seven"); break;
            case 8: printf(" eight"); break;
            case 9: printf(" nine"); break;
        }
    }

    printf("\n");
    return 0;
}
```

Результат виконання:

```
/home/mykolapoleiko/CLionProjects/lab/lab5.3
Enter three-digit number: 345
three hundred forty five

Process finished with exit code 0
```

```
/home/mykolapoleiko/CLionProjects/lab/lab5.3
Enter three-digit number: 311
three hundred eleven

Process finished with exit code 0
```

```
/home/mykolapoleiko/CLionProjects/lab/lab5.3
Enter three-digit number: 99
Number out of range
Process finished with exit code 1
```

```
/home/mykolapoleiko/CLionProjects/lab/lab5.3
Enter three-digit number: hello
Not a number
Process finished with exit code 1
```

Висновок

У ході виконання лабораторної роботи було вивчено умовні оператори мови C та набуто практичних навичок їх застосування. Оператор `?` використано для порівняння двох чисел: він дозволяє записати розгалуження компактно, а вкладені тернарні оператори — перевірити кілька умов поспіль. Конструкцію `if – else if – else` застосовано для визначення типу трикутника за сторонами та кутами; важливо, що гілки такої конструкції взаємовиключні, тому для одних вхідних даних виконується лише одна з них. Оператор `switch` використано для виведення числа прописом: він зручний, коли потрібно вибрати одну дію зі списку значень однієї змінної, а оператор `break` запобігає виконанню наступних гілок.